Exercise 14 — Design Pattern Implementation Challenge

Exercise Description

Select code exhibiting a design-pattern opportunity, analyse it, get implementation guidance, refactor to implement the pattern, **write/keep tests verifying behaviour is preserved**, and document the benefits.

Code chosen: `database_connection.py` (Python) — a `DatabaseConnection` whose `connect()` is a long `if db_type == 'mysql' / elif 'postgresql' / ...` chain. **Pattern applied: Factory.**

Files: - `database_connection_original.py` — untouched original. - `database_connection.py` — refactored to the Factory pattern. - `test_database_connection.py` — the original tests (unchanged), used to verify behaviour.

Why this is a Factory opportunity

The original `connect()` mixed two concerns: **choosing** a database type and the **per-type** connection-string logic. Every new database meant editing that one growing method — a classic Open/Closed Principle violation, and exactly the "complex object creation branching on a type" smell the Factory pattern addresses.

Implementation

- **`DatabaseConnector` (ABC):** the interface — `connect(config)`.
- **Concrete connectors:** `MySqlConnector`, `PostgreSqlConnector`, `MongoDbConnector`, `RedisConnector` — each owns only its own connection-string construction.
- **`ConnectorFactory`:** a registry `{db_type: connector_class}` with `create(db_type)` that returns the right connector or raises `ValueError` for unknown types.
- **`DatabaseConnection`:** unchanged public interface; `connect()` now prints the "Connecting..." banner, asks the factory for a connector, delegates, then prints "Connection successful!".

Keeping `DatabaseConnection`'s constructor and `connect()` signature identical meant the existing tests didn't need to change — the refactor is purely internal.

Verification — behaviour preserved

```
Ran 8 tests in 0.002s
OK
```

All eight tests pass, including the exact connection-string assertions (e.g. `mysql://…?charset=utf8`, `retryAttempts=3&poolSize=5`, `sslmode=require`) and the `ValueError` for an unsupported type.

Benefits gained

- **Open/Closed:** adding a new database = one new connector class + one registry line; `DatabaseConnection.connect()` never changes.
- **Single Responsibility:** each connector owns one database's quirks.
- **Testability:** each connector can be tested in isolation.
- **Readability:** the giant `if/elif` chain is gone; selection is a dictionary lookup.

Reflection Questions

1. **How did the pattern improve maintainability?** Type selection is now data (a registry), and each database's logic is isolated — changes stay local.
2. **What future changes are now easier?** Supporting a new database (e.g. SQLite/Cassandra), or swapping a connector's implementation, without touching the client or other connectors.
3. **Any unexpected challenges?** Preserving the **exact printed output and the pre-raise "Connecting…" banner** so the string-matching tests still pass — the factory must raise `ValueError` *before* "Connection successful!" is printed, mirroring the original control flow.

Submission for Exercise 14 — Design Pattern Implementation Challenge

1. Code selected & pattern identified: `database_connection.py` — a type-branching `connect()`; opportunity for the **Factory** pattern (complex object creation branching on a type).

2. Analysis & implementation guidance: the `if/elif` chain violates Open/Closed and mixes selection with per-type logic → introduce a connector interface, one concrete connector per database, and a factory/registry for selection.

3. Refactored code implementing the pattern: `database_connection.py` — `DatabaseConnector` ABC, four concrete connectors, `ConnectorFactory`, and a thin `DatabaseConnection.connect()` that delegates.

4. Tests verifying preserved behaviour: the original suite passes unchanged — **8 tests, OK** — including all connection-string assertions and the unsupported-type `ValueError`.

5. Benefits gained: Open/Closed extensibility (new DB = new class + registry entry), single-responsibility connectors, isolated testability, and a readable dictionary-lookup selection (reflection answers above).